



Drönarsvärm

Designspecifikation Drönarsvärm

2025/10/20

Designspecifikation

Adam Mejri, Edvard Wetind, Isac Widendahl
Linus Gustafsson, Oskar Haapaniemi, Oskar Herling, William Olsson

2025/10/20

Version 1.0



Drönarsvärm

Status

Granskad	Isac Widendahl	2025-10-16
Godkänd	Joel Wendin	2025-10-15



Projektidentitet

Grupp E-post: dronarsvarm.visionen-users@liuonline.onmicrosoft.com

Hemsida: <http://www.liu.se/grouppage>

Beställare: Joel Wendin, Reglerteknik, Linköpings universitet
E-post: joel.wendin@liu.se

Kund: Daniel Axehill, Linköpings Universitet
E-post: daniel.axehill@liu.se

Expertstöd: Martin Castor, Geistt/Inhumate
E-post: martin.castor@et.mil.se

Expertstöd: Erik Strid, Geistt/Inhumate
E-post: erik.strid@et.mil.se

Handledare: Joel Wikner
E-post: joel.wikner@liu.se

Kursansvarig: Daniel Axehill
E-post: daniel.axehill@liu.se

Projektdeltagare

Namn	Initialer	Ansvar	E-post
Adam Mejri	AM	Projektledare	adame022@student.liu.se
Edvard Wetind	EW	Mjukvaruansvarig	edvwe024@student.liu.se
Isac Widendahl	IW	Dokumentansvarig	isawi003@student.liu.se
Linus Gustafsson	LG	Informationsansvarig	lingu061@student.liu.se
Oskar Haapaniemi	OH	Simuleringsansvarig	oskha100@student.liu.se
Oskar Herling	OHE	Testansvarig	oskhe678@student.liu.se
William Olsson	WO	Designansvarig	wilol978@student.liu.se



INNEHÅLL

1	Inledning	1
1.1	Syfte och Mål	1
1.2	Begrepp	1
2	Översikt av systemet	1
2.1	Användargränssnitt	3
2.2	Kommunikationssystem	4
2.3	Uppdragsplanerare	4
2.4	Vägplanerare	4
2.5	Styrsystem	4
2.6	Simuleringsmiljö	4
2.7	Visionen	4
3	Implementation	5
3.1	Uppdragsplanerare	5
3.2	Vägplanerare	5
3.3	Styrsystem	7
3.4	Simuleringsmiljö	14
	Referenser	15

**DOKUMENTHISTORIK**

Version	Datum	Utförda ändringar	Utförda av	Granskad
1.0	2025-10-15	Tagit bort delkapitlet Ledarföljning från 3.3 Styrssystem, Ändrat tabell 2 och inledning i 3.3 Styrssystem, lagt till inledning till 3.3.3 Kollisionsundvikande	AM, EW, IW, LG, OH, OHE, WO	IW
0.2	2025-10-13	Andra utkast	AM, EW, IW, LG, OH, OHE, WO	IW
0.1	2025-10-03	Första utkast	AM, EW, IW, LG, OH, OHE, WO	IW



1 INLEDNING

Designspecifikationen ger en översikt över systemet i helhet samt mer detaljerade förklaringar för hur varje delsystem ska vara uppbyggd och hur dessa delsystem kommunicerar med varandra.

1.1 Syfte och Mål

Syftet med projektet är att projektmedlemmarna skall få erfarenhet att arbeta i grupp på ett sätt som liknar praktiskt ingenjörsarbete samt få mer domänspecifik kompetens inom svärmstyrning.

Målet är att utveckla en bas för styrning av drönarsvärmar som kan flyga och utföra uppdrag i formation samt som kan vidareutvecklas för att åstadkomma mer komplexa uppdrag.

1.2 Begrepp

Nedan defineras och förklaras frekvent använda begrepp.

Drönare - UAV av typen quadcopter

Drönarsvärm - En grupp på minst 50 drönare.

Centraldator - En dator som har kontakt med alla drönare med syfte att styra svärmen. I vår implementation kommer inte denna dator finnas utan vara simulerad.

GUI - Grafiskt användargränssnitt

Uppdrag - En uppgift given av centraldatorn till svärmen. Inkluderar exempelvis formationsflygning och att vissa drönare utför specialuppgifter som att patrullera ett område

Arena Visionen - En forskningsarena på Campus Valla, Linköpings Universitet. Utrymmet är bland annat utrustat med kamerabaserat lokaliseringssystem.

ORCA - Optical Reciprocal Collision Avoidance är en kollisionsundvikande algoritm som appliceras på system med flera entiteter som inte kommunicerar med varandra.

Synlighetsradie - Området som definierar vilka andra drönare som den relevanta drönaren kan kommunicera med.

2 ÖVERSIKT AV SYSTEMET

Projektets mål är att lägga grunden till utvecklingen av förmågan att med en storskalig drönarsvärm (ca 50 drönare) utföra flyguppgifter från start till landning genom decentraliserad styrning. Detta innefattar även formationsflygning och kollisionsundvikande manövrar. Projektet kommer även fokusera på uppdragsplanering och genomföring för drönarsvärmen. Systemet delas in i följande fem delsystem: Användargränssnitt, kommunikationssystem, uppdragsplane-

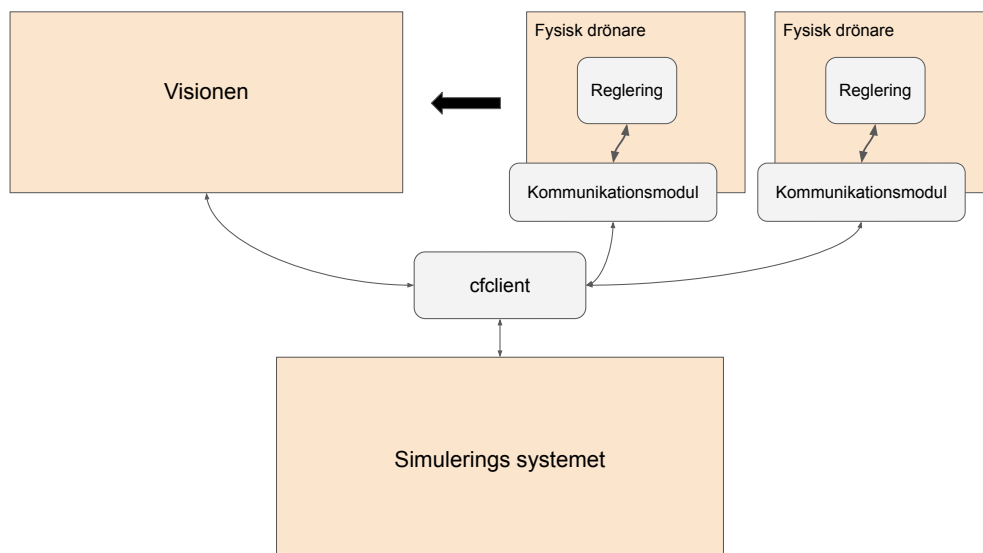


rare, vägplanerare och styrsystem. Systemet kommer i första hand att utvecklas i en avancerad simuleringsmiljö, men hela tiden med en fysisk implementering i sikte.

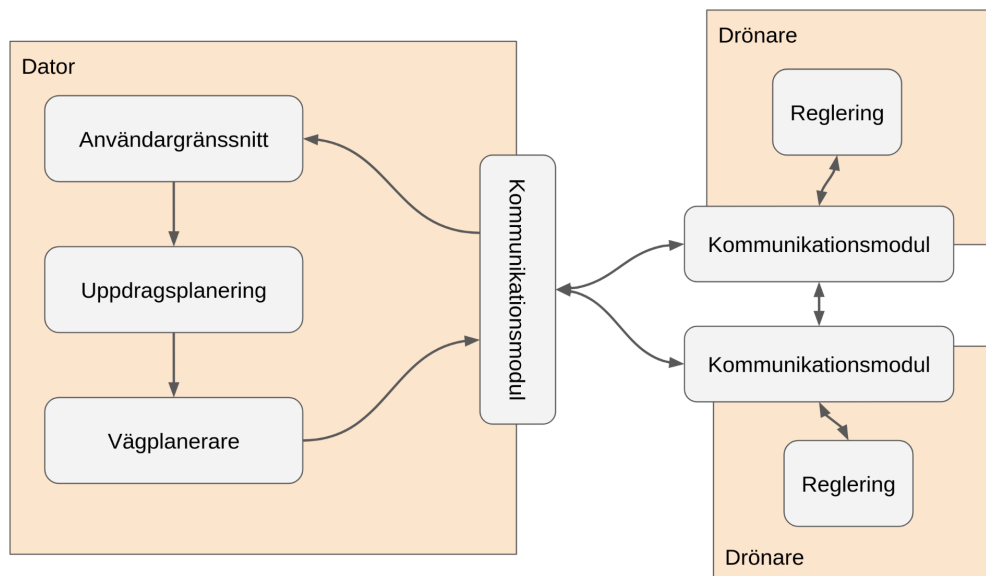
Ambitionen är att, om tiden tiden tillåter, skapa en brygga mellan den simulerade och den verkliga världen genom **ROS2**.

I praktiken innebär detta att ROS2 integreras direkt i simuleringsmiljön, vilket möjliggör kommunikation med *Arena Visionen* – en labsal utrustad med Qualisys infraröda kameror. Där kan fysiska drönare, försedda med reflekterande markörer, spåras i realtid. Varje drönare, både simulerad och fysisk, representeras som en egen ROS2-nod. Genom att Qualisys mäter och skickar positionerna vidare skapas ett gemensamt scenario där den fysiska drönaren kan interagera med den virtuella svärmen, som om de befann sig i samma luftrum.

Nedan illustreras både den simulerade miljön och det planerade gränssnittet mot *Arena Visionen*. Blocket titluerat **Simulerings systemet** i figur 1 finns mer detaljerat i figur 2.



Figur 1: Översiktliga blockscheman för implementation av systemet i både simulerad och fysisk miljö.



Figur 2: Översiktligt blockschema för implementation av systemet i simulering.

2.1 Användargränssnitt

Användargränssnittet bygger på ett tidigare utvecklat GUI vilket tillhandahålls av GEISTT och Inhumate. I användargränssnittet ska användaren kunna specificera uppdrag till drönarsvärmen. Variabler som begränsar svärmens beteende ska här bestämmas av användaren för att uppnå önskat resultat.

I nuläget existerar ett användargränssnitt med följande funktioner:

- Maximal altitud
- Minimal altitud
- Avsöknings zon
- Antal drönare i svärmen
- Avsöknings algoritm

Vid behov ska användargränssnittet kompletteras med ytterligare funktioner för att optimera utformningen av uppdrag.



2.2 Kommunikationssystem

Kommunikationssystemet består av programvara på varje drönare som tar emot position, hastighet och riktning från alla andra drönare inom synlighetsradien. Kommunikationssystemet ska även sköta all kommunikation mellan centraldatorn och drönarsvärmen.

I simuleringsmiljön kommer detta endast ske genom ett filter som gömmer alla drönare utanför synlighetsradien. För implementation på Crazyflies kommer det emuleras genom att informationen skickas via en centraldator då Crazyflies saknar kapaciteten att kommunicera med varandra.

I princip ska kommunikationssystemet kunna bytas ut till en mycket, rent kommunikationsmässigt, kraftfullare kommunikationsmodul och kunna kommunicera med många fler drönare.

2.3 Uppdragsplanerare

Uppdragsplanerarens uppgift är att ta in information från användargränssnittet och dela upp en uppgift i exekverbara delsteg. Dessa delsteg ska sedan skickas vidare till vägplaneraren.

2.4 Vägplanerare

Vägplanerarens uppgift är att ta emot information kring deluppdrag från uppdragsplaneraren och definiera konkreta rutter för drönarsvärmen att följa under uppdragets gång. Rutten ska innehålla information angående preliminär väg, formation och hastighet. Rutterna skickas till drönarsvärmen via kommunikationssystemet.

2.5 Styrsystem

Styrsystemet tar in data kring rutten från vägplaneraren via kommunikationssystemet. Med hjälp av regleralgoritmer styrs drönarna på individnivå i svärmen med målet att hålla formationen, följa den planerade rutten, undvika hinder samt avsöka ett område. Drönarna i svärmen ska undvika kollisioner med såväl omgivande terräng som andra drönare i svärmen.

2.6 Simuleringsmiljö

Simuleringen kommer göras i en Unity-baserad miljö skapad av GEISTT och Inhumate. I nuläget används Unity 2022.3.11f1 som simuleringsmiljö med tillhörande filer, vilka tillhandahålls av företaget GEISTT. Simuleringen använder de tidigare nämnda delsystemen för att visualisera drönarsvärmen samt för att genomföra tester. Testerna genomförs för att garantera att kraven som ställts på projektets delsystem uppnås. Vid implementering av riktiga drönare/fysiska objekt kommer dessa köras via ROS2 i Unity.

2.7 Visionen

Vi kommer visualisera simuleringen i laboratoriet Visionen. Det kommer göras genom att koppla upp simuleringen mot projektorer. Eventuellt kommer vi även implementera vår algoritm på Crazyflies så att de agerar som drönare i si-



mulationen. Ytterligare en möjlighet om tid finns är att med hjälp av Visionens positioneringssystem få den simulerade drönarsvärmen att interagera med en person.

3 IMPLEMENTATION

3.1 Uppdragsplanerare

Uppdragsplaneraren ska dela upp uppdraget i bestämda delar. Denna information skickas vidare som en kö till vägplaneraren så att den ska kunna iterera igenom alla delar. Tabell 1 visar ett exempel på hur ett uppdrag kan delas upp i olika delar. Ett typiskt avsökningsuppdrag där svärmen ska färdas till ett område, avsöka det, anta en given formation och sedan åka vidare, skickas då i en kö enligt följande *Färdas*, *Avsöka*, *Anta formation*, *Hålla formation under färd*

Namn	Data
Färdas	Start, Stop, Begränsningar på vägen.
Gå till formation	Formationens koordinater
Hålla formation under färd	Sekvens av koordinater
Avsöka	Yta som ska avsökas, Noggrannhet tills uppfyllt uppdrag, Avsöknings-algoritm

Tabell 1: Delar varje uppdrag delas upp i, samt deras prioritet

3.1.1 Områdesavsökning

När drönarsvärmen nått sitt mål kan ett uppdrag vara att där avsöka ett område utefter en given avsökningsalgoritm. Två sådana, Snake och PSO (Particle Swarm Optimization) finns redan implementerade. I mån av tid ska minst en av dessa decentraliseras.

Vilket område som ska avsökas väljs i användargränssnittet och kan vid simulering definieras i Unity.

3.2 Vägplanerare

Vägplanerarens uppdrag är att definiera en väg utifrån deluppdrag givna av uppdragsplaneraren. Då det ofta kan finnas hinder på vägen eller olika krav på vart man vill att drönarna ska åka som vägplaneraren behöver ta hänsyn till. Typiska begränsningar kan vara hålla sig på viss höjd, endast åka inom visst område eller att åka genom vissa punkter. Dessa begränsningar ges av uppdragsplaneraren och tas då i hänsyn när vägplaneraren försöker hitta en tillåten och tillräckligt effektiv väg. En referensväg behöver inte planeras för varje drönare individuellt utan varje drönare kommer att följa samma referens.

De algoritmer som kommer testas och utvärderas är RRT- och A*-algoritmen. En faktor som minskar behovet av en avancerad vägplanerare är det faktum att drönarna har implementerats med kollisionsundvikning. Vägplanerarens uppdrag blir därför att hitta en logisk väg över området med låg hänsyn till hinder längs vägen. Istället ska fokuset ligga på vilka områden drönaren ska flyga över och vilka områden som ska undvikas.



3.2.1 RRT-algoritmen

RRT eller rapidly exploring random tree [1] som algoritmen också kallas, skapar ett träd som expanderar över ett område tills trädet når målet. Först skapas en slumpmässig nod där algoritmen sedan undersöker vilken nod i trädet som ligger närmast denna. Om inget hinder blockerar vägen skapas en ny nod med ett bestämt avstånd i riktning mot den slumpmässiga noden och läggs till i trädet. Detta itereras tills en ny nod når målet. Algoritmen beskrivs enligt följande.

Algorithm 1 Pseudokod för RRT-algoritm.

```
1: Input:  $q_I$  (startposition),  $N$  (antal iterationer)
2: Output:  $G = (V, E)$  (träd av möjliga vägar)
3:  $V \leftarrow \{q_I\}$ ,  $E \leftarrow \emptyset$ 
4: for  $i = 1 \dots N$  do
5:    $q_{rand} \leftarrow \text{Sample}()$ 
6:    $q_{nearest} \leftarrow \text{Nearest}(G = (V, E), q_{rand})$ 
7:    $q_{new} \leftarrow \text{Steer}(q_{nearest}, q_{rand})$ 
8:   if  $\text{ObstacleFree}(q_{nearest}, q_{new})$  then
9:      $V \leftarrow V \cup \{q_{new}\}$ 
10:     $E \leftarrow E \cup \{(q_{nearest}, q_{new})\}$ 
11:   end if
12: end for
13: return  $G = (V, E)$ 
```



3.2.2 A*-algoritmen

A*-algoritmen, utvecklad och först presenterad i *A Formal Basis for the Heuristic Determination of Minimum Cost Paths* [2], börjar med att ta in en lista med noder som den ska undersöka. Därefter går den igenom noderna tills den når målet enligt följande algoritm.

Algorithm 2 Pseudokod för A*-algoritm.

```
1: Input:  $x_I$  (startnod),  $x_G$  (målnod)
2: Output: Sökresultat: SUCCESS om en väg hittas, annars FAILURE
3:  $C(x_I) \leftarrow 0$ 
4:  $Q.insert(x_I, C(x_I) + h(x_I))$ 
5: while  $Q \neq \emptyset$  do
6:    $x \leftarrow Q.pop()$ 
7:   if  $x = x_G$  then
8:     return SUCCESS
9:   end if
10:  for  $u \in U(x)$  do
11:     $x' \leftarrow f(x, u)$ 
12:    if ingen tidigare kostnad för  $x'$  eller  $C(x') > C(x) + d(x, x')$  then
13:       $previous(x') \leftarrow x$ 
14:       $C(x') \leftarrow C(x) + d(x, x')$ 
15:       $Q.insert(x', C(x') + h(x'))$ 
16:    end if
17:  end for
18: end while
19: return FAILURE
```

3.3 Styrsystem

Styrsystemet finns implementerat på varje drönare och ska följa vägen som vägplaneraren skapar. Algoritmer för styrsystemet tillämpas på tre olika områden, formationsflygning, vägföljning och undvikande av hinder. Tabell 2 nedan innehåller de algoritmer som används inom respektive område.

Namn	Argument
Formationsflygning	Boids algoritm
Vägföljning	Heuristikbaserad vektorföljning
Undvikande av hinder	Artificial potential field, Optimal Reciprocal Collision Avoidance (ORCA)

Tabell 2: Exempel på styrsystem för reglering baserat på situation

I avsnittet beskrivs olika algoritmer med olika komplexitet, grunden i vår lösning är en boids-algoritm som beskrivs i 3.3.1, där ges en tydligare definition av boids men grunden är att vi skapar tre vektorer som vi väger mot varandra för att styra drönarna med syfte att hålla ihop svärmen. Vi behöver komplettera boids med förmåga att färdas till ett mål och att undvika hinder, idéer för det beskrivs i 3.3.2 och 3.3.3. Tanken är att generera en fjärde och femte vektor som vägs mot boids vektorer.

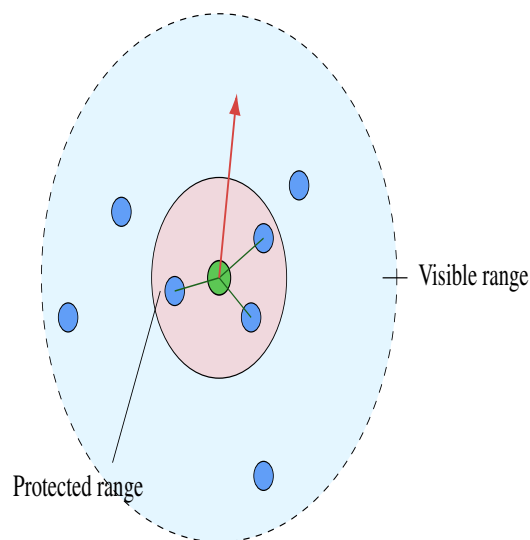


3.3.1 boids-algoritm

För att få drönarsvärmen att flyga samordnat men utan att tvingas in i en strikt formation används en boids-algoritm. Algoritmen utvecklades av Craig Reynolds [3] och är inspirerad av hur flockar av djur, såsom fåglar eller fiskstim, rör sig koordinerat utan central styrning. I en boid-modell styr varje enskild individ ("boid") sig själv, men tar hänsyn till andra närliggande individer. Genom enkla lokala regler uppstår ett kollektivt flockbeteende där gruppen rör sig samordnat samtidigt som individerna undviker hinder (både andra boider och objekt i omgivningen). För att implementera boid-beteendet används vanligtvis tre grundläggande regler: **Separation**, **Alignment** och **Cohesion**

Separation

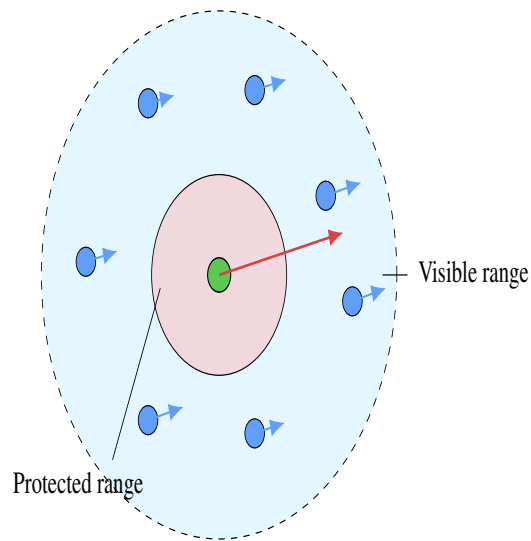
Separation syftar till att undvika kollisioner genom att hålla avstånd till närliggande drönare. Runt varje individ definieras en så kallad *protected range*, ett skyddat område som andra inte får befinna sig inom. Om en annan individ kommer in i detta område genereras en styrvektor som pekar bort från den inträngande individen, se figur 3 nedan.



Figur 3: Separation: undvik grannar inom *protected range*. Röd pil visar styrning bort från den lokala överbelastningen.

Alignment

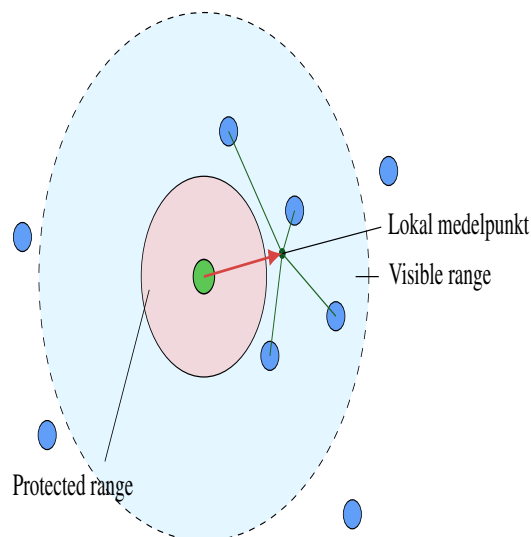
Alignment ser till att svärmen rör sig i samma riktning genom att varje drönare anpassar sin hastighetsvektor efter de individer som ligger inom dess *visible range* (där *visible range* > *protected range*). Drönaren beräknar medelhastigheten för alla grannar inom detta synfält och justerar sin egen hastighetsvektor mot detta medelvärde. Resultatet blir att drönarna successivt synkroniserar riktning och hastighet, vilket ger ett koordinerat rörelsemönster, se figur 4 nedan.



Figur 4: Alignment: anpassa kurs mot grannarnas medelriktning inom *visible range*.

Cohesion

Cohesion syftar till att hålla ihop flocken genom att varje drönare långsamt rör sig mot den lokala medelpunkten för grannarna inom dess visible range. Detta görs genom att beräkna medelpositionen för alla grannar och generera en styrvektor som pekar mot denna punkt. Cohesion gör att flocken håller ihop utan att tvingas in i en specifik formation, se figur 5 nedan.



Figur 5: Cohesion: mjukstyrning mot medelposition för lokala boids.

**Algorithm 3** Pseudokod för Boids i 3D med Separation, Alignment och Cohesion.

```

1: Input: Flock  $\mathcal{B}$ , synlighetsradie  $R_{\text{vis}}$ , skyddad radie  $R_{\text{prot}}$ , vikter  $w_{\text{sep}}, w_{\text{ali}}, w_{\text{coh}}$ , tidssteg  $\Delta t$ , maxhastighet  $v_{\text{max}}$ 
2: Output: Uppdaterade positioner  $\mathbf{x}_i \in \mathbb{R}^3$  och hastigheter  $\mathbf{v}_i \in \mathbb{R}^3$  för varje boid  $i$ 
3: for each boid  $i \in \mathcal{B}$  do
4:    $\mathbf{s} \leftarrow \mathbf{0}$  ▷ Separation
5:    $\mathbf{a} \leftarrow \mathbf{0}$  ▷ Alignment
6:    $\mathbf{c} \leftarrow \mathbf{0}$  ▷ Cohesion
7:    $n \leftarrow 0$  ▷ Antal grannar
8:   for each boid  $j \in \mathcal{B}, j \neq i$  do
9:      $\mathbf{d} \leftarrow \mathbf{x}_j - \mathbf{x}_i$ 
10:     $r \leftarrow \|\mathbf{d}\|$ 
11:    if  $r < R_{\text{vis}}$  then
12:       $n \leftarrow n + 1$ 
13:      if  $0 < r < R_{\text{prot}}$  then
14:         $\mathbf{s} \leftarrow \mathbf{s} - \mathbf{d}/r$  ▷ Separation: repellera om för nära
15:      end if
16:       $\mathbf{a} \leftarrow \mathbf{a} + \mathbf{v}_j$  ▷ Alignment: summera grannars hastigheter
17:       $\mathbf{c} \leftarrow \mathbf{c} + \mathbf{x}_j$  ▷ Cohesion: summera grannars positioner
18:    end if
19:  end for
20:  if  $n > 0$  then
21:     $\mathbf{a} \leftarrow \mathbf{a}/n - \mathbf{v}_i$  ▷ Alignment: justera mot medelhastighet
22:     $\mathbf{c} \leftarrow \mathbf{c}/n - \mathbf{x}_i$  ▷ Cohesion: flytta mot centroid
23:  end if
24:   $\mathbf{u}_i \leftarrow w_{\text{sep}}\mathbf{s} + w_{\text{ali}}\mathbf{a} + w_{\text{coh}}\mathbf{c}$  ▷ Samlad acceleration från tre regler
25:   $\mathbf{v}_i \leftarrow \mathbf{v}_i + \mathbf{u}_i\Delta t$  ▷ Integrera hastighet och position
26:  if  $\|\mathbf{v}_i\| > v_{\text{max}}$  then
27:     $\mathbf{v}_i \leftarrow v_{\text{max}} \cdot \mathbf{v}_i / \|\mathbf{v}_i\|$  ▷ Begränsa hastighet
28:  end if
29:   $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i\Delta t$ 
30: end for

```

3.3.2 Globala styrprinciper

För att svärmen inte enbart ska flyga i formation utan även kunna styras mot ett mål implementeras en global styrprincip, nämligen Heuristikbaserad vektorföljning.

Heuristikbaserad vektorföljning

I denna metod följer varje drönare inte en specifik ledare, utan en global riktning som beräknas utifrån det euklidiska avståndet mellan startpunkt och mål. Denna riktning fungerar som en heuristisk styrvektor som alla drönare i svärmen justerar sin rörelse mot, oberoende av de andra drönarnas positioner. På så sätt får varje drönare en individuell men samordnad rörelse mot målet utan behov av central styrning eller ledarutnämning.



Den globala styrvektorn kan exempelvis genereras genom att beräkna vektorn $\vec{g} = \vec{G} - \vec{S}$ mellan startpunkt $\vec{S}$ och målpunkt $\vec{G}$, som sedan normeras och används som riktning för hela svärmen. Varje drönare väger därefter samman denna globala riktning med sitt lokala boids-beteende (separation, alignment och cohesion) vilket gör att svärmen både bibehåller formation och kontinuerligt rör sig mot målet. Detta beskrivs i pseudokod nedan.

Algorithm 4 Pseudokod för målriktad hastighetsvektor (per drönare) utifrån euklidiskt avstånd.

```

1: Input: Flock  $\mathcal{B}$ , mål  $\mathcal{G} = \{\mathbf{g}_i\}$ , proportionalitet  $k_{\text{goal}}$ , vikt  $w_{\text{goal}}$ , tidssteg  $\Delta t$ , maxhastighet  $v_{\text{max}}$ , stoppradie  $\varepsilon$ 
2: Output: Uppdaterade positioner  $\mathbf{x}_i \in \mathbb{R}^3$  och hastigheter  $\mathbf{v}_i \in \mathbb{R}^3$  för varje drönare  $i$ 
3: for each drönare  $i \in \mathcal{B}$  do
4:    $\mathbf{d} \leftarrow \mathbf{g}_i - \mathbf{x}_i$                                 ▷ Riktningssvektor mot mål
5:    $r \leftarrow \|\mathbf{d}\|$                                 ▷ Euklidiskt avstånd till mål
6:   if  $r \leq \varepsilon$  then
7:      $\mathbf{v}^{\text{des}} \leftarrow \mathbf{0}$                                 ▷ Stanna i målområdet
8:   else
9:      $\hat{\mathbf{d}} \leftarrow \mathbf{d}/r$                                 ▷ Normaliserad riktning
10:     $v^* \leftarrow \min\{k_{\text{goal}} r, v_{\text{max}}\}$                 ▷ Hastighetsprofil  $\propto$  avstånd, begränsad av  $v_{\text{max}}$ 
11:     $\mathbf{v}^{\text{des}} \leftarrow v^* \hat{\mathbf{d}}$                                 ▷ Önskad hastighetsvektor mot mål
12:  end if
13:   $\mathbf{a}_i \leftarrow w_{\text{goal}}(\mathbf{v}^{\text{des}} - \mathbf{v}_i)$                 ▷ Enkel hastighetsreglering (P på hastighet)
14:   $\mathbf{v}_i \leftarrow \mathbf{v}_i + \mathbf{a}_i \Delta t$ 
15:  if  $\|\mathbf{v}_i\| > v_{\text{max}}$  then
16:     $\mathbf{v}_i \leftarrow v_{\text{max}} \cdot \mathbf{v}_i / \|\mathbf{v}_i\|$                 ▷ Begränsa hastighet
17:  end if
18:   $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i \Delta t$ 
19: end for

```

3.3.3 Kollisionsundvikande

Utöver drönar-drönar-kollisioner som kan ske men som tas om hand av boid algoritmen kan drönare även kollidera med omgivningen som exempelvis träd eller byggnader. Nedan beskrivs två olika algoritmer för att sådana undvika kollisioner, *Optimal Reciprocal Collision Avoidance (ORCA)* samt *Artificial potential field*.

Optimal Reciprocal Collision Avoidance

Optimal Reciprocal Collision Avoidance (ORCA) introducerar en algoritm för att undvika kollisioner mellan agenter som rör sig i samma område samt med statiska objekt. Metoden är *decentraliserad*, linjär i sin geometriska formulering och skalbar, vilket möjliggör säker navigering även i trånga miljöer. ORCA presenterades i *Reciprocal n-body Collision Avoidance* [4].

Algoritmen bygger på att varje agent har en önskad hastighetsvektor baserad på sitt mål (t.ex. att nå en punkt eller följa en bana, såsom genererad av boids-algoritmen i avsnitt 3.3.1). Innan agenten uppdaterar sin hastighet förutser den potentiella kollisioner inom en given tidshorisont, baserat på nuvarande hastigheter hos sig själv och andra objekt eller agenter.

För varje potentiell kollision definieras en geometrisk begränsning i form av ett halvrum i hastighetsrummet. Alla agenter gör motsvarande beräkningar och delar på ansvaret för att undvika kollisionen, vilket innebär att varje agent



justerar sin hastighet halvvägs bort från den potentiella kollisionsriktningen. Slutligen väljer agenten den kollisionsfria hastighetsvektor som ligger närmast den önskade. Detta visas i pseudokod nedan.

Algorithm 5 Pseudokod för ORCA för 3D-drönarsvärn

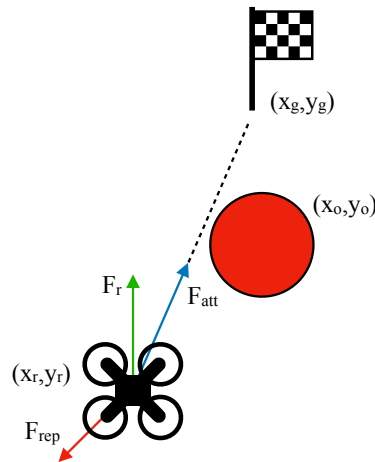
```

1: Input: Drönarens position  $p_i$ , hastighet  $v_i$ , radie  $R_i$ , maxhastighet  $v_{\max}$ , maxacceleration  $a_{\max}$ , tidshorisont  $\tau$ ,
   sensorradie  $r_{\text{sense}}$ , tidssteg  $\Delta t$ 
2: Output: Ny kollisionsfri hastighet  $v_i^{\text{new}}$  och uppdaterad position  $p_i$ 
3: for each drönare  $i$  do
4:    $\mathcal{N}_i \leftarrow \text{NearbyAgents}(i, r_{\text{sense}})$ 
5:    $\mathcal{O}_i \leftarrow \text{NearbyObstacles}(i, r_{\text{sense}})$ 
6:    $P \leftarrow \emptyset$  ▷ Mängd ORCA-plan  $(a_k, n_k)$ 
7:   for each  $j \in \mathcal{N}_i$  do
8:      $p_{ij} \leftarrow p_j - p_i$ 
9:      $v_{ij} \leftarrow v_j - v_i$ 
10:     $R \leftarrow R_i + R_j$ 
11:    if  $\|p_{ij}\| > R$  then
12:       $w \leftarrow v_{ij} - p_{ij}/\tau$ 
13:       $n \leftarrow \text{VO\_normal}(p_{ij}, w)$ 
14:       $u \leftarrow (R/\tau - w \cdot n) n$ 
15:       $a \leftarrow v_i + \frac{1}{2}u$  ▷ Reciprokt undvikande
16:    else
17:       $n \leftarrow p_{ij} / \|p_{ij}\|$ 
18:       $u \leftarrow (R - \|p_{ij}\|) / \Delta t n$ 
19:       $a \leftarrow v_i + \frac{1}{2}u$ 
20:    end if
21:     $P \leftarrow P \cup \{(a, n)\}$ 
22:  end for
23:  for each hinder  $o \in \mathcal{O}_i$  do
24:     $(n, u) \leftarrow \text{ObstacleORCA3D}(i, o, \tau)$ 
25:     $a \leftarrow v_i + u$ 
26:     $P \leftarrow P \cup \{(a, n)\}$ 
27:  end for
28:   $v_{\text{pref}} \leftarrow \text{DesiredVelocity}(i)$ 
29:   $v_{\text{goal}} \leftarrow \text{ClipNorm}(v_{\text{pref}}, v_{\max})$ 
30:   $v_i^{\text{new}} \leftarrow \text{SolveORCA3D}(P, v_{\text{goal}}, v_{\max})$ 
31:   $v_i^{\text{new}} \leftarrow \text{LimitSlewRate}(v_i, v_i^{\text{new}}, a_{\max}, \Delta t)$ 
32:   $p_i \leftarrow p_i + v_i^{\text{new}} \Delta t$ 
33:   $v_i \leftarrow v_i^{\text{new}}$ 
34: end for

```

**Artificial potential field**

För undvikande av hinder kommer artificiellt potentialfält [5] eller också kallat APF användas och testas. Denna algoritmen skapar en attraktiv och en repulsiv kraft från olika punkter i omgivningen. Punkten som skapar den attraktiva kraften F_{att} är i det här fallet målet, (x_g, y_g) , och kommer dra drönaren, (x_r, y_r) , mot sig. Punkten som skapar den repulsiva kraften F_{rep} är i det här fallet hindret, (x_o, y_o) , och kommer skjuta drönaren från sig. På detta sätt skapas den resulterande kraften F_r som leder drönaren runt hindret och sedan fortsatt mot målet. Genom att låta alla drönare i svärmen självständigt använda sig av denna algoritmen kommer formationen bibehållas samtidigt som drönarna var för sig undviker de hinder som uppkommer. Se figur 7 samt pseudokod nedan.

**Figur 6:** Visualisering av APF-algoritmen

Algoritmen beskrivs enligt följande.

Algorithm 6 Pseudokod för APF-algitm.

```

1: Input: Startposition  $(x_r, y_r)$ , hinderposition  $\vec{o} = (x_o, y_o)$ , målnod  $\vec{g} = (x_g, y_g)$ 
2: Output: Resulterande kraftvektor  $\vec{F}_r$ 
3:  $(x_r, y_r) \leftarrow (0, 0)$ 
4:  $\vec{o} \leftarrow (x_o, y_o)$ 
5:  $\vec{g} \leftarrow (x_g, y_g)$ 
6: if NearObstacle() then
7:    $\vec{F}_{rep} \leftarrow -\vec{o} / \|\vec{o}\|^2$                                 ▷ Repulsiv kraft från hinder
8:    $\vec{F}_{att} \leftarrow \vec{g} / \|\vec{g}\|$                                 ▷ Attraktiv kraft mot mål
9:    $\vec{F}_r \leftarrow \vec{F}_{rep} + \vec{F}_{att}$                                 ▷ Total kraftvektor
10: end if
11: return  $\vec{F}_r$ 

```

Vad som är ett hinder definieras i simuleringen med hjälp av metoden **Raycast** som hör till Unitys Physics-klass, vilken i praktiken skickar ut en stråle från drönaren mot målet och märker om något står i vägen.



3.4 Simuleringsmiljö

Samtlig kod i delsystemen och simuleringsmiljön skrivs i programmeringsspråket C#. Delsystemen kommer att delas upp i mindre moduler där varje modul är ett eget C#-script i Unity. Drönarna i Unity är `GameObjects` med komponenterna `Rigidbody` och `BoxCollider`. Huvudscriptet kopplat till respektive drönare kallas för `Quadcopter`. I detta script definieras modellparametrar enligt figur 8 nedan.

Limits	
Max Climb Acceleration	10
Max Planar Acceleration	10
Max Yaw Acceleration	5
Max Climb Rate	10
Max Descent Rate	10
Max Speed	20
Max Yaw Rate	5
Velocity Tilt	1
Charge	
Discharge Per Acceleration	0.0001
Range Per Charge	1
Reserve Charge	10

Figur 7: Modellparametrar för drönarmodellen i Unity

Övriga script kopplade till svärm- och uppdragsspecifika beteenden kopplas även till den enskilda drönarmodellen.



REFERENSER

- [1] S. M. LaValle and J. J. K. Jr, “Randomized kinodynamic planning,” *The International Journal of Robotics Research*, vol. 20, no. 5, pp. 378–400, 2001.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] C. W. Reynolds, “Flocks, herds, and schools: A distributed behavioral model,” in *Proc. 14th Annu. Conf. on Computer Graphics and Interactive Techniques (SIGGRAPH '87)*. Anaheim, CA, USA: ACM, 1987, pp. 25–34. [Online]. Available: <https://dl.acm.org/doi/10.1145/37401.37406>
- [4] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n-body collision avoidance,” in *Robotics Research: The 14th International Symposium ISRR*, ser. Springer Tracts in Advanced Robotics, C. Pradalier, R. Siegwart, and G. Hirzinger, Eds. Berlin, Heidelberg: Springer, 2011, vol. 70, pp. 3–19. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-642-19457-3_1
- [5] K. Elferink, “Cooperative platoon maneuvering using artificial potential fields,” Master’s thesis, Eindhoven University of Technology, 2016.